

Лабораторная работа № 8

Мониторинг и анализ производительности Windows Server

Продолжительность

90 минут.

Среда выполнения

- Hyper-V;
 - Windows Server;
 - PowerShell;
 - Event Viewer;
 - Performance Monitor;
 - сервер FS01;
 - рабочее место администратора ADMIN01 — при наличии.
-

1. Цель работы

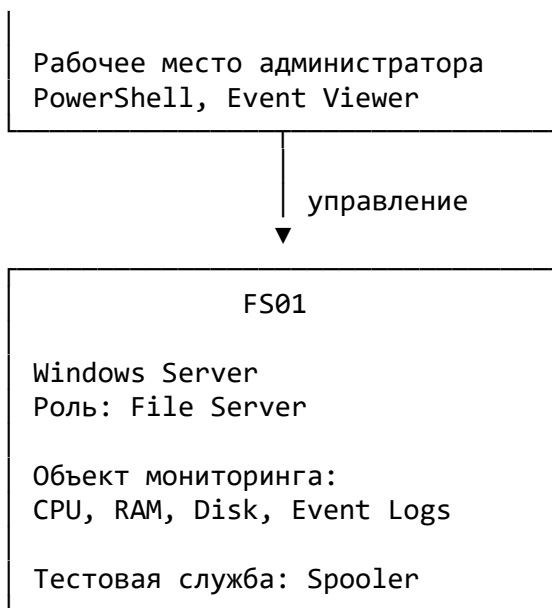
Научиться выполнять базовый мониторинг Windows Server и связывать пользовательский симптом с измерениями и событиями системы.

В ходе основной части необходимо:

- проверить исходное состояние сервера;
 - определить его роль и условия измерения;
 - выполнить быструю проверку процессов;
 - собрать короткий baseline CPU, RAM и диска;
 - сохранить результаты измерений;
 - получить ошибки и предупреждения из Event Log;
 - смоделировать безопасный отказ службы;
 - найти соответствующие события;
 - восстановить службу;
 - подтвердить возврат к штатному состоянию;
 - оформить краткий диагностический вывод.
-

2. Схема стенда

ADMIN01



Работу можно выполнять непосредственно на FS01, если отдельный ADMIN01 не подготовлен.

3. Исходные условия

До начала работы преподаватель подготавливает:

- тестовый сервер FS01;
- установленную Windows Server;
- отдельную учебную виртуальную машину;
- локальную или доменную административную учётную запись;
- Snapshot виртуальной машины;
- не менее 2 ГБ свободного места на системном диске;
- работающую службу Spooler.

В лаборатории не используются:

- контроллер домена как основной объект нагрузки;
 - Exchange Server;
 - Hyper-V Host;
 - производственный файловый сервер;
 - критичные службы организации.
-

4. Основные понятия

Термин	Значение
--------	----------

Термин	Значение
Monitoring	Постоянное наблюдение за состоянием сервера
Diagnostics	Поиск причины уже обнаруженной проблемы
Baseline	Зафиксированное нормальное поведение системы
Utilization	Степень использования ресурса
Saturation	Очередь, ожидание или нехватка ресурса
Errors	Ошибки оборудования, драйверов или программ
Counter	Измеряемый показатель производительности
Event	Запись о действии, ошибке или изменении состояния

5. Правила безопасности

- Все тесты выполняются только на учебной виртуальной машине.
 - Нельзя останавливать AD DS, DNS, Exchange, Hyper-V Management или файловые службы рабочего сервера.
 - Нельзя заполнять системный диск до нуля.
 - Нельзя запускать бесконечную нагрузку без ограниченной продолжительности.
 - Нельзя удалять журналы событий до анализа.
 - Перезагрузка не используется как первый способ диагностики.
 - После каждого теста обязательно выполняется восстановление и повторная проверка.
-

6. Базовые задания

Задание 1. Проверить исходное состояние FS01

Открываем PowerShell от имени администратора.

Проверяем имя сервера:

```
hostname  
hostname -f
```

Ожидается:

```
FS01  
FS01.company.test
```

Проверяем операционную систему:

```
Get-ComputerInfo |  
    Select-Object WindowsProductName,  
        WindowsVersion,
```

```
OsBuildNumber,  
CsName
```

Проверяем роль сервера:

```
Get-WindowsFeature |  
  Where-Object InstallState -eq 'Installed' |  
  Sort-Object Name |  
  Format-Table DisplayName, Name
```

В отчёте указываем:

```
Server: FS01  
Role: File Server  
Period: учебное измерение  
Load: без активной пользовательской нагрузки
```

Задание 2. Подготовить каталог результатов

Создаём рабочий каталог:

```
New-Item `  
  -ItemType Directory `  
  -Path C:\Lab\Monitoring `  
  -Force |  
  Out-Null
```

Проверяем:

```
Get-Item C:\Lab\Monitoring
```

Все CSV- и TXT-файлы лабораторной работы сохраняются в этом каталоге.

Задание 3. Выполнить быструю проверку ресурсов

Проверяем процессор и память:

```
Get-CimInstance Win32_ComputerSystem |  
  Select-Object Name,  
    NumberOfLogicalProcessors,  
    @{  
      Name='RAM_GB'  
      Expression={  
        [math]::Round(  
          $_.TotalPhysicalMemory / 1GB,  
          2  
        )  
      }  
    }  
  }
```

Проверяем тома:

```
Get-Volume |  
  Sort-Object DriveLetter |  
  Format-Table DriveLetter,  
    FileSystemLabel,  
    HealthStatus,  
    Size,  
    SizeRemaining
```

Проверяем сетевые адаптеры:

```
Get-NetAdapter |  
  Format-Table Name,  
    Status,  
    LinkSpeed,  
    MacAddress
```

Проверяем статистику сети:

```
Get-NetAdapterStatistics |  
  Format-Table Name,  
    ReceivedBytes,  
    SentBytes,  
    ReceivedPacketErrors,  
    OutboundPacketErrors
```

Задание 4. Найти процессы с наибольшим потреблением ресурсов

Проверяем процессы по накопленному CPU Time:

```
Get-Process |  
  Sort-Object CPU -Descending |  
  Select-Object -First 10 `Name,  
    Id,  
    CPU,  
    WorkingSet,  
    Handles |  
  Format-Table -AutoSize
```

Проверяем процессы по объёму памяти:

```
Get-Process |  
  Sort-Object WorkingSet -Descending |  
  Select-Object -First 10 `Name,  
    Id,  
    @{  
      Name='WorkingSetMB'  
      Expression={  
        [math]::Round(  
          ($_.WorkingSet / 1MB)        )  
      }  
    }
```

```

        $_.WorkingSet / 1MB,
        2
    )
}
},
CPU |
Format-Table -AutoSize

```

Важно

Поле CPU показывает накопленное процессорное время процесса, а не мгновенный процент загрузки.

Для анализа нагрузки во времени используются Performance Counters.

Задание 5. Проверить доступные Performance Counters

На разных языковых версиях Windows названия счётчиков могут отличаться.

Проверяем список наборов:

```

Get-Counter -ListSet * |
    Select-Object CounterSetName |
    Sort-Object CounterSetName

```

Проверяем счётчики процессора:

```

Get-Counter -ListSet Processor |
    Select-Object -ExpandProperty Paths

```

Если сервер использует локализованные названия, преподаватель указывает фактические пути счётчиков для данного образа Windows.

Задание 6. Собрать короткий baseline

Создаём набор счётчиков:

```

$Counters = @(
    '\Processor(_Total)\% Processor Time'
    '\System\Processor Queue Length'
    '\Memory\Available MBytes'
    '\Memory\% Committed Bytes In Use'
    '\PhysicalDisk(_Total)\Avg. Disk sec/Read'
    '\PhysicalDisk(_Total)\Avg. Disk sec/Write'
)

```

Собираем 12 измерений с интервалом 5 секунд:

```

$Baseline = Get-Counter `
    -Counter $Counters `

```

```
-SampleInterval 5 `
-MaxSamples 12
```

Общая продолжительность измерения — около одной минуты.

Это короткая учебная проверка, а не полноценный суточный baseline.

Задание 7. Сохранить измерения baseline

Преобразуем результаты в таблицу:

```
$BaselineTable = $Baseline.CounterSamples |
    Select-Object Timestamp,
        Path,
        CookedValue
```

Выводим:

```
$BaselineTable |
    Format-Table -AutoSize
```

Сохраняем CSV:

```
$BaselineTable |
    Export-Csv `
        C:\Lab\Monitoring\baseline.csv `
        -NoTypeInfoation `
        -Encoding UTF8
```

Проверяем файл:

```
Get-Item C:\Lab\Monitoring\baseline.csv
```

Задание 8. Рассчитать средние и максимальные значения

Группируем результаты по счётчику:

```
$BaselineSummary = $BaselineTable |
    Group-Object Path |
    ForEach-Object {
        $Values = $_.Group.CookedValue

        [PSCustomObject]@{
            Counter = $_.Name
            Average = [math]::Round(
                (
                    $Values |
                    Measure-Object -Average
                ).Average,
                3
            )
        }
    }
```

```

    )
    Maximum = [math]::Round(
        (
            $Values |
            Measure-Object -Maximum
        ).Maximum,
        3
    )
    Samples = $Values.Count
}
}

```

Выводим:

```

$BaselineSummary |
    Format-Table -AutoSize

```

Сохраняем:

```

$BaselineSummary |
    Export-Csv `
        C:\Lab\Monitoring\baseline-summary.csv `
        -NoTypeInfoation `
        -Encoding UTF8

```

В отчёте нельзя делать вывод только по одному максимальному значению. Необходимо учитывать среднее значение и длительность отклонения.

Задание 9. Проверить события System за последние 24 часа

Получаем Critical и Error:

```

$SystemErrors = Get-WinEvent `
    -FilterHashtable @{
        LogName     = 'System'
        Level       = 1,2
        StartTime   = (Get-Date).AddHours(-24)
    } `
    -ErrorAction SilentlyContinue |
    Select-Object TimeCreated,
        Id,
        ProviderName,
        LevelDisplayName,
        Message

```

Выводим последние события:

```

$SystemErrors |
    Select-Object -First 20 |
    Format-Table -Wrap

```

Сохраняем:

```
$SystemErrors |  
  Export-Csv `C:\Lab\Monitoring\system-errors.csv `  
  -NoTypeInfoation `  
  -Encoding UTF8
```

При анализе события фиксируем

- Log Name;
- Time Created;
- Provider;
- Event ID;
- Level;
- Message.

Event ID без Provider не является достаточным объяснением ошибки.

Задание 10. Проверить дисковые события

Ищем предупреждения и ошибки дисковой подсистемы:

```
$DiskEvents = Get-WinEvent `  
  -FilterHashtable @{  
    LogName      = 'System'  
    ProviderName = 'disk','storport','Ntfs'  
    Level        = 1,2,3  
    StartTime    = (Get-Date).AddHours(-24)  
  } `  
  -ErrorAction SilentlyContinue |  
  Select-Object TimeCreated,  
    Id,  
    ProviderName,  
    LevelDisplayName,  
    Message
```

Выводим:

```
$DiskEvents |  
  Format-Table -Wrap
```

Пустой результат допустим и означает, что за указанный период выбранные providers не записали Critical, Error или Warning.

Сохраняем:

```
$DiskEvents |  
  Export-Csv `C:\Lab\Monitoring\disk-events.csv `  
  -NoTypeInfoation `  
  -Encoding UTF8
```

Задание 11. Проверить состояние тестовой службы

В качестве безопасной тестовой службы используется:

Print Spooler

Проверяем:

```
Get-Service Spooler |  
    Format-List Name,  
                DisplayName,  
                Status,  
                StartType
```

Ожидается:

Status : Running

Записываем время начала теста:

```
$FailureStart = Get-Date  
$FailureStart
```

Задание 12. Смоделировать отказ службы

Останавливаем только тестовую службу:

```
Stop-Service Spooler
```

Проверяем:

```
Get-Service Spooler
```

Ожидается:

Status : Stopped

Пользовательский симптом

При остановленной службе:

- печать недоступна;
- очередь печати не обслуживается;
- остальные роли сервера продолжают работать.

Не останавливаем критичные службы сервера.

Задание 13. Найти событие изменения службы

Проверяем события Service Control Manager после начала теста:

```
$ServiceEvents = Get-WinEvent `
-FilterHashtable @{
    LogName        = 'System'
    ProviderName   = 'Service Control Manager'
    StartTime      = $FailureStart.AddMinutes(-1)
} `
-ErrorAction SilentlyContinue |
Where-Object {
    $_.Message -match 'Print Spooler|Spooler'
} |
Select-Object TimeCreated,
    Id,
    ProviderName,
    LevelDisplayName,
    Message
```

Выводим:

```
$ServiceEvents |
    Format-Table -Wrap
```

Сохраняем:

```
$ServiceEvents |
    Export-Csv `
        C:\Lab\Monitoring\spooler-events.csv `
        -NoTypeInfoation `
        -Encoding UTF8
```

В зависимости от способа остановки и версии Windows могут появиться информационные события изменения состояния службы.

События аварийного завершения обычно связаны с другими Event ID, чем штатная остановка.

Задание 14. Восстановить службу

Запускаем:

```
Start-Service Spooler
```

Проверяем:

```
Get-Service Spooler
```

Ожидается:

Status : Running

Записываем время восстановления:

```
$RecoveryTime = Get-Date  
$RecoveryTime
```

Задание 15. Выполнить повторную проверку

Проверяем службу:

```
Get-Service Spooler
```

Проверяем основные счётчики:

```
$RecoveryCounters = Get-Counter `
    -Counter $Counters `
    -SampleInterval 5 `
    -MaxSamples 3
```

Сохраняем:

```
$RecoveryCounters.CounterSamples |  
    Select-Object Timestamp,  
        Path,  
        CookedValue |  
    Export-Csv `
        C:\Lab\Monitoring\recovery-counters.csv `
        -NoTypeInfo `
        -Encoding UTF8
```

Проверяем события после восстановления:

```
Get-WinEvent `
    -FilterHashtable @{  
        LogName      = 'System'  
        ProviderName = 'Service Control Manager'  
        StartTime    = $FailureStart  
    } `
    -ErrorAction SilentlyContinue |  
Where-Object {  
    $_.Message -match 'Print Spooler|Spooler'  
} |  
Select-Object TimeCreated,  
    Id,  
    LevelDisplayName,  
    Message |  
Format-Table -Wrap
```

Задание 16. Сформировать диагностическую таблицу

Заполняем таблицу:

Этап	Результат
Симптом	Служба печати недоступна
Время начала	
Метрики	CPU, RAM и Disk не показывают критичного изменения
События	Service Control Manager, фактический Event ID
Гипотеза	Служба Spooler остановлена
Проверка	Get-Service Spooler
Действие	Start-Service Spooler
Время восстановления	
Повторная проверка	Служба имеет статус Running

Пример вывода

Симптом:
служба печати недоступна.

Метрики:
CPU, память и дисковая задержка находятся около baseline.

События:
в журнале System обнаружено событие Service Control Manager, соответствующее изменению состояния Spooler.

Гипотеза:
проблема вызвана остановленной службой, а не нехваткой ресурсов.

Действие:
служба Spooler запущена.

Результат:
служба работает, повторная проверка выполнена.

7. Ожидаемый результат базовой части

К концу лабораторной работы:

- определена роль FS01;
- собран минутный baseline;
- рассчитаны средние и максимальные значения;
- результаты сохранены в CSV;
- выгружены ошибки System;
- проверены дисковые events;
- смоделирована остановка безопасной службы;

- найдено связанное событие;
 - служба восстановлена;
 - выполнена повторная проверка;
 - сформирована диагностическая цепочка.
-

8. Дополнительные задания

Дополнительное задание 1. Смоделировать ограниченную CPU-нагрузку

Тест выполняется только на учебной виртуальной машине.

Записываем время:

```
$CpuTestStart = Get-Date
```

Запускаем ограниченную нагрузку на 60 секунд:

```
$CpuLoadJob = Start-Job {  
    $EndTime = (Get-Date).AddSeconds(60)  
  
    while ((Get-Date) -lt $EndTime) {  
        [math]::Sqrt(  
            (Get-Random -Minimum 1 -Maximum 1000000)  
        ) | Out-Null  
    }  
}
```

Во время нагрузки измеряем CPU:

```
Get-Counter `'  
    '\Processor(_Total)\% Processor Time' `'  
    -SampleInterval 5 `'  
    -MaxSamples 12
```

Проверяем завершение задания:

```
Wait-Job $CpuLoadJob  
Receive-Job $CpuLoadJob  
Remove-Job $CpuLoadJob
```

Повторно измеряем CPU после остановки.

Сравниваем:

- baseline;
- нагрузку;
- восстановление.

Один PowerShell Job может не загрузить все логические процессоры. Его задача — показать изменение показателя, а не довести сервер до полной перегрузки.

Дополнительное задание 2. Создать Data Collector Set

На FS01 открываем:

Server Manager
→ Tools
→ Performance Monitor

Переходим:

Data Collector Sets
→ User Defined
→ New
→ Data Collector Set

Название:

FS01-Baseline

Выбираем:

Create manually
→ Performance counter

Добавляем:

\Processor(_Total)\% Processor Time
\System\Processor Queue Length
\Memory\Available MBytes
\Memory\% Committed Bytes In Use
\Memory\Pages/sec
\PhysicalDisk(_Total)\Avg. Disk sec/Read
\PhysicalDisk(_Total)\Avg. Disk sec/Write

Устанавливаем:

Sample Interval: 5 seconds
Log Format: Binary
Path: D:\PerfLogs\FS01-Baseline

Запускаем DCS на 5–10 минут.

Проверяем созданный BLG-файл.

Открываем его через:

Performance Monitor
→ View Log Data
→ Log files

Дополнительное задание 3. Создать Custom View

Открываем:

Event Viewer
→ Custom Views
→ Create Custom View

Создаём представление:

Service Failures

Параметры:

Параметр	Значение
Log	System
Level	Error
Source	Service Control Manager
Event ID	7031, 7034

Сохраняем представление.

Дополнительно создаём:

Disk Warnings

С параметрами:

Параметр	Значение
Log	System
Level	Critical, Error, Warning
Sources	disk, Ntfs, storport

Дополнительное задание 4. Проанализировать дисковую задержку

Собираем:

```
$DiskCounters = @(
    '\PhysicalDisk(_Total)\Avg. Disk sec/Read'
    '\PhysicalDisk(_Total)\Avg. Disk sec/Write'
    '\PhysicalDisk(_Total)\Current Disk Queue Length'
    '\LogicalDisk(C:)\% Free Space'
    '\LogicalDisk(C:)\Free Megabytes'
)

Get-Counter `
-Counter $DiskCounters `
```

```
-SampleInterval 5 `
-MaxSamples 12
```

Задержка возвращается в секундах.

Примеры преобразования:

```
0.005 sec = 5 ms
0.020 sec = 20 ms
0.100 sec = 100 ms
```

Универсальный порог не используется. Результат сравнивается с типом диска, ролью сервера и baseline.

Дополнительное задание 5. Проанализировать сеть

Проверяем статистику адаптера:

```
Get-NetAdapterStatistics |
    Format-Table Name,
        ReceivedBytes,
        SentBytes,
        ReceivedPacketErrors,
        OutboundPacketErrors,
        ReceivedDiscardedPackets,
        OutboundDiscardedPackets
```

Собираем счётчики:

```
Get-Counter `
    '\Network Interface(*)\Bytes Total/sec' `
    -SampleInterval 5 `
    -MaxSamples 12
```

Сравниваем трафик со скоростью интерфейса:

```
Get-NetAdapter |
    Format-Table Name,
        Status,
        LinkSpeed
```

Нормальная скорость передачи не исключает ошибки и отброшенные пакеты.

Дополнительное задание 6. Настроить WEF/WEC

На сервере WEC01 выполнить:

```
wecutil qc
```

Подтвердить первоначальную настройку Event Collector.

Проверить службу:

`Get-Service Wecsvc`

На источнике проверить WinRM:

`winrm quickconfig`

Создать тестовую подписку на события:

System
Service Control Manager
Event ID 7031, 7034

Проверить появление событий в журнале:

Forwarded Events

WEF/WEC используется для централизованного сбора событий, но не заменяет полноценную SIEM-систему.

9. Типовые ошибки

Симптом	Вероятная причина	Что проверить
Get-Counter не находит путь	Локализованное имя счётчика	<code>Get-Counter -ListSet</code>
CSV пустой	Нет событий по фильтру	Период и Provider
Много случайных ошибок	Слишком широкий фильтр	Log, Provider, Level, Time
Не найдено событие службы	Штатная остановка не создаёт 7031	Проверить все SCM Events
Spooler не запускается	Зависимость или конфигурация службы	<code>Get-Service</code> , Event Log
CPU высокий только секунду	Краткий пик	Длительность и несколько samples
Высокий Pages/sec	Не обязательно нехватка RAM	Available RAM и commit
Диск заполнен	Недостаточно места для логов	<code>Get-Volume</code>

10. Способы проверки

Состояние сервера

`Get-ComputerInfo`
`Get-Volume`

```
Get-NetAdapter
Get-Service
```

Процессы

```
Get-Process |
    Sort-Object CPU -Descending |
    Select-Object -First 10
```

Performance Counters

```
Get-Counter `
    '\Processor(_Total)\% Processor Time'
```

События

```
Get-WinEvent `
    -FilterHashtable @{
        LogName = 'System'
        Level   = 1,2
    } `
    -MaxEvents 20
```

Тестовая служба

```
Get-Service Spooler
```

11. Приёмочная матрица

№	Проверка	Ожидаемый результат	Результат
1	FS01	Сервер доступен	
2	Роль	Зафиксирована	
3	Каталог отчёта	Создан	
4	Процессы CPU	Получен Топ 10	
5	Процессы RAM	Получен Топ 10	
6	Baseline	12 измерений	
7	Baseline CSV	Создан	
8	Summary	Average и Maximum	
9	System Events	Выгружены	
10	Disk Events	Проверены	
11	Spooler до теста	Running	
12	Spooler во время теста	Stopped	
13	SCM Events	Найдены или объяснены	
14	Spooler после теста	Running	
15	Recovery Counters	Собраны	

№	Проверка	Ожидаемый результат	Результат
16	Диагностическая цепочка	Сформирована	

12. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему стенда.
3. Роль сервера.
4. Условия измерения.
5. Сведения о CPU и RAM.
6. Сведения о дисках.
7. Тор 10 процессов по CPU.
8. Тор 10 процессов по памяти.
9. Перечень выбранных счётчиков.
10. Таблицу Average и Maximum.
11. Файл или фрагмент `baseline.csv`.
12. Критичные события System.
13. Результат проверки дисковых событий.
14. Состояние Spooler до отказа.
15. Время начала теста.
16. Состояние службы во время отказа.
17. События Service Control Manager.
18. Выполненное действие.
19. Состояние после восстановления.
20. Заполненную диагностическую таблицу.
21. Приёмочную матрицу.
22. Краткий вывод.

Для дополнительных заданий включаются только фактически выполненные результаты.

13. Чек-лист выполнения

Базовая часть

- ☐ Проверено имя FS01.
- ☐ Определена роль сервера.
- ☐ Создан каталог результатов.
- ☐ Проверены CPU и RAM.

- ☐ Проверены диски.
- ☐ Проверены сетевые адаптеры.
- ☐ Получен список процессов по CPU.
- ☐ Получен список процессов по RAM.
- ☐ Проверены доступные counters.
- ☐ Собран короткий baseline.
- ☐ Результаты экспортированы в CSV.
- ☐ Рассчитаны Average и Maximum.
- ☐ Получены Critical и Error из System.
- ☐ Проверены дисковые events.
- ☐ Проверена служба Spooler.
- ☐ Смоделирована остановка службы.
- ☐ Найдены связанные события.
- ☐ Служба восстановлена.
- ☐ Выполнена повторная проверка.
- ☐ Сформирована диагностическая цепочка.

Дополнительная часть

- ☐ Смоделирована CPU-нагрузка.
 - ☐ Выполнено сравнение до, во время и после нагрузки.
 - ☐ Создан Data Collector Set.
 - ☐ Создан BLG-файл.
 - ☐ Создан Custom View.
 - ☐ Проанализирована Disk Latency.
 - ☐ Проанализирована сеть.
 - ☐ Настроен WEF/WEC.
-

14. Контрольные вопросы

1. Чем Monitoring отличается от Diagnostics?
2. Что такое Baseline и почему одного измерения недостаточно?
3. Какие показатели входят в модель USE?
4. Почему Event ID нужно рассматривать вместе с Provider?
5. Чем накопленное поле CPU процесса отличается от % Processor Time?
6. Почему высокий Pages/сек не всегда означает нехватку RAM?
7. В каких единицах измеряется Avg. Disk sec/Read?
8. Почему после восстановления службы нужно повторно проверить метрики и события?